Live Site Deploy Guide

Lecture Scientifique / Lecture Portal - how we connect to production and push updates from a local Windows development machine.

Document date: August 2026

1. Overview

Production is not updated by uploading a cPanel archive manually for day-to-day changes. The working process used in this project is:

- Build the React SPA locally into the Laravel public folder.
- Connect over SSH/SFTP to the DigitalOcean VPS.
- Upload a lean code archive (app + SPA assets), preserving live .env and storage.
- Run Artisan migrate / cache / optional workshop sync on the server.

The live site URL is https://lecturescientifique.ca. The application root on the server is /var/www/lecture-scientifique/backend (document root serves backend/public).

2. Connection details

Item	Value
Protocol	SSH + SFTP (Paramiko from Windows)
Host / IP	146.190.248.212
SSH user	root
App path	/var/www/lecture-scientifique/backend
Public URL	https://lecturescientifique.ca
Web server	Nginx (+ PHP-FPM)

Security: Do not commit the server password into git, PDFs that are shared publicly, or chat logs. Store the root password in a password manager. Scripts used for deploy should read the password from an environment variable (e.g. LECTURE_DEPLOY_PASSWORD) or a local ignored file.

3. Prerequisites on your PC

- Project checked out locally (e.g. C:\xampp\htdocs\Lecture Portal).
- Node.js 18+ for the Teacher Dashboard Vite build.
- PHP + Composer available if you refresh vendor locally.
- Python 3 with the paramiko package (pip install paramiko).
- Network access to the VPS on port 22.

4. Step-by-step deploy process

Step 1: Make and test your changes locally

Develop against local XAMPP / Vite (http://localhost:5173 with API proxy to the Laravel backend). Confirm workshops, admin UI, and APIs work before shipping.

Step 2: Build the production SPA

From the Teacher Dashboard folder, run the deploy build. This writes app.html and the hashed assets into backend/public/ (via Vite outDir + scripts/copy-spa-build.mjs).

```
cd "Teacher Dashboard"
npm install          # first time only
npm run build:deploy
```

Verify these exist before uploading:

- backend/public/app.html
- backend/public/assets/ (JS/CSS/images)
- backend/vendor/autoload.php (for full deploys that include vendor)

Step 3: Prepare a lean deploy package

Package only what should overwrite production. Typical include list:

- app/, bootstrap/, config/, database/, lang/, resources/, routes/
- public/ (especially app.html + assets/) - exclude public/storage media
- vendor/, artisan, composer.json, composer.lock (when dependencies changed)

Always exclude:

- .env and .env.* (never overwrite production secrets)
- storage/ (uploads, logs, framework cache - keep server copy)
- node_modules/, .git/, tests/, public/spa/

Step 4: Connect with SSH/SFTP

From a temporary Python deploy script (or any SSH client), connect to the host as root using Paramiko. Example pattern:

```
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(
    "146.190.248.212",
    username="root",
    password=os.environ["LECTURE_DEPLOY_PASSWORD"],
    timeout=30,
    allow_agent=False,
    look_for_keys=False,
)
```

Step 5: Backup live .env and storage, then upload

On the server, copy .env and storage aside. Upload the tar.gz over SFTP to /tmp, extract into the app directory, then restore .env and storage.

```
# On server (via SSH)
cp /var/www/lecture-scientifique/backend/.env /tmp/lecture.env.bak
```

```
cp -a /var/www/lecture-scientifique/backend/storage /tmp/lecture-storage-bak

# After SFTP upload of archive to /tmp/lecture-code-update.tar.gz
tar -xzf /tmp/lecture-code-update.tar.gz -C /var/www/lecture-scientifique/backend
cp /tmp/lecture.env.bak /var/www/lecture-scientifique/backend/.env
cp -a /tmp/lecture-storage-bak/. /var/www/lecture-scientifique/backend/storage/
chown -R www-data:www-data storage bootstrap/cache
```

Step 6: Run migrations and clear caches

Execute Artisan from the backend directory on the server:

```
cd /var/www/lecture-scientifique/backend
php artisan migrate --force
php artisan storage:link || true
php artisan route:clear
php artisan config:clear
php artisan cache:clear
php artisan view:clear
php artisan config:cache
rm -f bootstrap/cache/routes*.php
```

Removing stale route cache files avoids cases where new API routes exist in code but are not registered until the cache is cleared.

Step 7: Optional - sync the 15 platform workshops

When assessment templates or workshop cards must be reset/published on live:

```
cd /var/www/lecture-scientifique/backend
php artisan assessments:sync-platform-workshops --replace
```

This reads database/data/platform-workshop-assessments.json (bundled with the deploy), archives existing platform assessments, and creates Workshops 1-15 with format_1...format_15 and grade All (Assign to All).

Step 8: Verify and clean up

Confirm the site loads, admin workshops look correct, and APIs respond. Delete temporary deploy scripts and /tmp archives from the server. Hard-refresh the browser (Ctrl+F5) so the new SPA assets load.

5. Typical Python deploy script flow

The Cursor agent / local one-shot scripts follow this sequence:

- Build archive of selected backend paths with tarfile (gzip).
- SSH connect -> backup .env + storage.
- SFTP put archive -> extract -> restore .env + storage.
- Run migrate, optional assessments:sync-platform-workshops --replace.
- Clear/rebuild config cache; remove routes*.php cache files.
- Remove /tmp archive and local temp deploy script.

6. What not to overwrite on live

Path	Why keep server copy
.env	DB password, APP_KEY, mail, domain settings
storage/app/public	Uploaded content library & media
storage/logs	Production diagnostics

7. Quick reference commands

Local build

```
cd "Teacher Dashboard" && npm run build:deploy
```

SSH probe (manual)

```
ssh root@146.190.248.212
```

Server app directory

```
cd /var/www/lecture-scientifique/backend
```

8. Related project files

- DEPLOY.md - original cPanel / archive install notes (alternate path).
- Teacher Dashboard/package.json - build:deploy script.
- scripts/copy-spa-build.mjs - copies Vite build into backend/public.
- backend/app/Console/Commands/SyncPlatformWorkshops.php - workshop sync.
- backend/database/data/platform-workshop-assessments.json - workshop payloads.

9. Troubleshooting

- Blank page: rebuild SPA; ensure public/app.html and public/assets exist on server.
- Old UI after deploy: hard refresh; confirm new hashed JS files under public/assets.
- API route missing: clear route/config cache; remove bootstrap/cache/routes*.php.
- 500 errors: check storage/logs/laravel.log; fix storage permissions for www-data.
- SSH auth failure: confirm IP, user root, and current password from the password manager.

End of guide. Keep credentials out of source control. Prefer short-lived deploy scripts that read secrets from the environment.